

☐ I2C 통신용 필수 패키지 설치

- 가상환경이 아닌 일반 터미널에서 실행. 새로운 터미널 열어서 작업.

```
$ sudo apt install -y i2c-tools python3-smbus
```

- IMU 센서(MPU6050)와 통신하려면 I2C 관련 시스템 도구가 필수적

```
pi@zerotoai:~/ai_rover $ sudo apt install -y i2c-tools python3-smbus
i2c-tools is already the newest version (4.4-2).
i2c-tools set to manually installed.
python3-smbus is already the newest version (4.4-2).
python3-smbus set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
pi@zerotoai:~/ai_rover $
```

8) [실습] 라즈베리파이 가상환경(venv) 구축 및 라이브러리 설치

라즈베리파이 OS 12버전 이후부터는 시스템 안정성을 위해, pip 설치를 OS 전역(Global)에 함부로 설치하지 못함(PEP 668 정책). 따라서 우리는 AI-Rover 프로젝트만을 위한 독립적인 공간, 즉 가상환경(venv)을 만들고 그곳에 필요한 라이브러리 설치하고 실행, vs code에서 진행.

① vs code로 SSH 접속 (pp. 33 참고)

> SSH: 192.168.137.216

- ☐ vs code 맨 아래 왼쪽에서 접속 상태 확인, IP주소는 다를수 있음.

② 사용자 폴더 이동, AI-Rover 프로젝트를 진행할 폴더로 이동

```
$ cd ~ # ~ 사용자 폴더로 이동
$ cd ai_rover # ai_rover 폴더 이동
```

③ 가상환경 생성 : ai_rover_venv (교재에서 딱 1번만 생성.)

```
$ python3 -m venv ai_rover_venv --system-site-packages
```

- ☐ --system-site-packages 옵션은 매우 중요 이 마법의 옵션을 붙여주면, 우리가 앞서 설치했던 패키지(i2c-tools 등)를 가상환경 안에서도 그대로 사용

④ 가상 환경 활성화(사용할때마다 실행 ★)

```
$ source ~/ai_rover/ai_rover_venv/bin/activate
```

- 가상 환경이 활성화되면 프롬프트 앞에 (ai_rover_venv)와 같이 가상 환경 이름이 표시
- 이 작업은 라즈베리파이를 켜다 켜 때마다, 혹은 새 터미널을 열 때마다 코드 실행 전에 반드시 수행해야 함

```
pi@zerotoai:~ $ cd ai_rover/
pi@zerotoai:~/ai_rover $ python3 -m venv ai_rover_venv --system-site-packages
pi@zerotoai:~/ai_rover $ source ~/ai_rover/ai_rover_venv/bin/activate
(ai_rover_venv) pi@zerotoai:~/ai_rover $
```

- ☐ 터미널 프롬프트 해석하기

```
(ai_rover_venv) pi@zeroToAI: ~/ai_rover $
```

- (ai_rover_venv) pi@zeroToAI: ~/ai_rover \$: 작동중인 가상환경 이름 (이게 보여야 정상)
- (ai_rover_venv) pi@zeroToAI: ~/ai_rover \$: 현재 사용자 계정 이름
- (ai_rover_venv) pi@zeroToAI: ~/ai_rover \$: 라즈베리파이 기기 이름(호스트명)
- (ai_rover_venv) pi@zeroToAI: ~/ai_rover \$: 사용자 홈 폴더 (/home/pi)
- (ai_rover_venv) pi@zeroToAI: ~/ai_rover \$: 현재 위치한 폴더

- 이후 표기는 아래와 같이 프롬프트 앞에 가상환경 이름만 표기.

```
(ai_rover_venv) $
```

⑤ 가상환경 비활성화 : deactivate

```
(ai_rover_venv) $ deactivate
```

```
● (ai_rover_venv) pi@zerotoai:~/ai_rover $ deactivate
○ pi@zerotoai:~/ai_rover $ █
```

⑥ 생성한 가상환경 폴더 완전 삭제

이름이 잘못되었거나 필요 없는 가상환경 폴더 및 파일 완전 삭제하는 방법.

☐ 가상환경 폴더 확인

- 소스 코드 폴더로 이동

```
$ cd ~/ai_rover
```

- 가상환경 폴더 확인

```
$ ls
```

ai_rover_venv가 보아야 함

☐ 가상환경 폴더(ai_rover_venv) 삭제

```
$ rm -rf ai_rover_venv
```

- r : 폴더 안에 있는 폴더 및 파일 모두 삭제
- f : 강제 삭제, 즉 확인절차 없이 바로 삭제

☐ 가상환경 활성화

```
$ source ~/ai_rover/ai_rover_venv/bin/activate
```

☐ 패키지 도구 pip 업데이트 (설치 오류 방지)

```
(ai_rover_venv) $ pip install --upgrade pip
```

```
● (ai_rover_venv) pi@zerotoai:~/ai_rover $ pip install --upgrade pip
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple
Requirement already satisfied: pip in ./ai_rover_venv/lib/python3.13/site-packages (25.1.1)
Collecting pip
  Downloading https://www.piwheels.org/simple/pip/pip-26.1.1-py3-none-any.whl (1.8 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 1.3 MB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.1.1
    Uninstalling pip-25.1.1:
      Successfully uninstalled pip-25.1.1
  Successfully installed pip-26.1.1
○ (ai_rover_venv) pi@zerotoai:~/ai_rover $ █
```

☐ 하드웨어 제어 및 센서 통신 라이브러리 설치

```
(ai_rover_venv) $ pip install gpiozero pyserial pynmea2 mpu6050-raspberrypi
```

```

• (ai_rover_venv) pi@zerotoai:~/ai_rover $ pip install gpiozero pyserial pynmea2 mpu6050-raspberrypi
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple
Requirement already satisfied: gpiozero in /usr/lib/python3/dist-packages (2.0.1)
Requirement already satisfied: pyserial in /usr/lib/python3/dist-packages (3.5)
Collecting pynmea2
  Downloading pynmea2-1.19.0-py3-none-any.whl (30 kB)
Collecting mpu6050-raspberrypi
  Downloading mpu6050_raspberrypi-1.2-py3-none-any.whl (6.5 kB)
Requirement already satisfied: colorzero in /usr/lib/python3/dist-packages (from gpiozero) (2.0)
Installing collected packages: pynmea2, mpu6050-raspberrypi
Successfully installed mpu6050-raspberrypi-1.2 pynmea2-1.19.0
○ (ai_rover_venv) pi@zerotoai:~/ai_rover $

```

□ 컴퓨터 비전(OpenCV) 라이브러리 설치

```
(ai_rover_venv) $ pip install opencv-contrib-python
```

```

• (ai_rover_venv) pi@zerotoai:~/ai_rover $ pip install opencv-contrib-python
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple
Collecting opencv-contrib-python
  Downloading opencv_contrib_python-4.13.0.92-cp37-abi3-manylinux_2_28_aarch64.whl.metadata (19 kB)
Requirement already satisfied: numpy>=2 in /usr/lib/python3/dist-packages (from opencv-contrib-python) (2.2.4)
Downloading opencv_contrib_python-4.13.0.92-cp37-abi3-manylinux_2_28_aarch64.whl (52.5 MB)
 52.5/52.5 MB 2.8 MB/s 0:00:18
Installing collected packages: opencv-contrib-python
Successfully installed opencv-contrib-python-4.13.0.92
○ (ai_rover_venv) pi@zerotoai:~/ai_rover $

```

□ 임시 작업 폴더 생성 (설치시 용량 부족 현상 방지, ★ 꼭 하셔야합니다.)

- SD 카드의 홈 폴더에 넉넉한 임시 폴더 만들
- 라즈베리파이의 기본 임시 폴더인 /tmp는 2G 용량뿐임.
- AI 추론 및 엣지 라이브러리들은 설치시 매우 큰 임시공간이 필요함

```
(ai_rover_venv) $ mkdir ~/pip_tmp
```

□ 환경변수(TMPDIR)에 위에서 만든 임시 폴더로 바꿔서 설치

; 명령어 맨 앞에 TMPDIR=~/pip_tmp를 붙여주면, 해당 폴더를 임시 작업장으로 사용하여 2GB 제한 없이 쾌적하게 설치됩니다.

; 캐시를 남기지 않는 --no-cache-dir 옵션도 추가

```
(ai_rover_venv) $ TMPDIR=~/pip_tmp pip install --no-cache-dir 설치패키지이름
```

□ AI 추론 및 엣지 최적화 라이브러리 설치 (시간 정말 많이 소요됨 40분~1시간)

```
(ai_rover_venv) $ TMPDIR=~/pip_tmp pip install --no-cache-dir torch torchvision ultralytics onnx onnxruntime
```

- torch, torchvision : 현존하는 가장 강력한 인공지능 프레임워크인 PyTorch
로봇의 딥러닝 연산을 담당.
- ultralytics : 표지판이나 신호등, 사람을 눈 깜짝할 새에 찾아낼 YOLO(You Only Look Once)
알고리즘의 최신 라이브러리입니다.
- opencv-python: 방금 2장에서 우리가 카메라를 제어할 때 썼던, 영상 처리의 세계 표준

```

• (ai_rover_venv) pi@zerotoai:~/ai_rover $ TMPDIR=~/pip_tmp pip install --no-cache-dir torch torchvision ultralytics onnx onnxruntime
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple
Collecting torch
  Downloading torch-2.11.0-cp313-cp313-manylinux_2_28_aarch64.whl.metadata (29 kB)
Collecting torchvision
  Downloading torchvision-0.26.0-cp313-cp313-manylinux_2_28_aarch64.whl.metadata (5.5 kB)
Collecting ultralytics
  Downloading ultralytics-8.4.46-py3-none-any.whl (1.2 MB)
 1.2/1.2 MB 1.3 MB/s 0:00:01

```

```
Installing collected packages: nvidia-cusparse-cu13, mpmath, flatbuffers, cuda-toolkit, triton, sympy, protobuf, polars-runtime-32, opencv-
python, nvidia-nvtx, nvidia-nvshmem-cu13, nvidia-nvjitlink, nvidia-nccl-cu13, nvidia-curand, nvidia-cufile, nvidia-cuda-runtime, nvidia-cuda-
nrtc, nvidia-cuda-cupti, nvidia-cublas, networkx, ml_dtypes, fsspec, filelock, cuda-pathfinder, polars, onnxruntime, onnx, nvidia-cuspars-
e-cu13, nvidia-cudnn-cu13, cuda-bindings, nvidia-cusolver, torch, ultralytics-thop, torchvision, ultralytics
Successfully installed cuda-bindings-13.2.0 cuda-pathfinder-1.5.4 cuda-toolkit-13.0.2 filelock-3.29.0 flatbuffers-20181003210633 fsspec-2026.
4.0 ml_dtypes-0.5.4 mpmath-1.3.0 networkx-3.6.1 nvidia-cublas-13.1.0.3 nvidia-cuda-cupti-13.0.85 nvidia-cuda-nrtc-13.0.88 nvidia-cuda-runtir
e-13.0.96 nvidia-cudnn-cu13-9.19.0.56 nvidia-cufft-12.0.0.61 nvidia-cufile-1.15.1.6 nvidia-curand-10.4.0.35 nvidia-cusolver-12.0.4.66 nvidia-
cuspars-12.6.3.3 nvidia-cusparselt-cu13-0.8.0 nvidia-nccl-cu13-2.28.9 nvidia-nvjitlink-13.0.88 nvidia-nvshmem-cu13-3.4.5 nvidia-nvtx-13.0.85
onnx-1.21.0 onnxruntime-1.25.1 opencv-python-4.13.0.92 polars-1.40.1 polars-runtime-32-1.40.1 protobuf-7.34.1 sympy-1.14.0 torch-2.11.0 torc
hvision-0.26.0 triton-3.6.0 ultralytics-8.4.46 ultralytics-thop-2.0.19
(ai_rover_venv) pi@zerotoai:~/ai_rover $
```

☐ 설치 끝난 후 임시 폴더(pip_tmp) 완전 삭제하기

```
(ai_rover_venv) $ rm -rf ~/pip_tmp
```

```
(ai_rover_venv) pi@zerotoai:~/ai_rover $ rm -rf ~/pip_tmp
(ai_rover_venv) pi@zerotoai:~/ai_rover $ ls
ai_rover_venv  ch02_vision_basics  ch04_gps_imu_nav  ch06_state_machin  ch08_custom_training  ch10_multi_threading
ch01_warmingup  ch03_lane_tracking  ch05_yolo_detection  ch07_hybrid_driving  ch09_onnx_optimization  ch11_appendix_guide
(ai_rover_venv) pi@zerotoai:~/ai_rover $
```

☐ 라이브러리 설치 및 버전 확인

```
(smart_factory_venv) $ pip show torch
(smart_factory_venv) $ pip show ultralytics
(smart_factory_venv) $ pip show onnx
```

- show 명령어는 설치가 잘 되어 있는지 여부만 알 수 있음. 실제 동작 여부는 모름.

☐ 라이브러리 동작 확인

```
(smart_factory_venv) $ python3
>>> import torch
>>> import ultralytics
>>> import cv2
>>> exit()
```

- 결과 확인: 명령어를 쳤을 때 붉은색 에러 메시지 없이 다음 줄(>>>)이 부드럽게 나오면 완벽하게 성공한 것입니다! 확인이 끝났으면 exit()를 입력하여 파이썬 모드를 빠져나옵니다.

☐ Tip : 설치 시 발생할 수 있는 문제와 해결책 (Troubleshooting)

설치 시간이 너무 오래 걸려요!

AI 라이브러리(특히 torch, ultralytics)는 용량이 매우 큼니다. 라즈베리파이의 인터넷 속도나 모델 사양에 따라 설치에만 30분~1시간 이상 걸릴 수 있습니다. 화면이 멈춘 것이 아니니, 여유를 가지고 기다려 주세요.

opencv-contrib-python 설치 후 코드 실행 시 에러가 나요.

만약 OpenCV를 불러올 때(import cv2) 공유 라이브러리(libGL.so.1 등)를 찾을 수 없다는 에러가 뜬다면, 다음 명령어로 OS 그래픽 패키지를 추가 설치해 주면 깔끔하게 해결됩니다. `sudo apt install -y libgl1-mesa-glx`

torch 설치 중 프로세스가 죽어요 (Killed)

라즈베리파이의 RAM 메모리가 부족하면 가끔 설치가 강제로 종료(Killed)됩니다. 이럴 때는 다른 프로그램(웹 브라우저 등)을 모두 끄고, 라즈베리파이의 온도가 너무 높지 않은지 확인한 후 다시 시도해 보세요."